PPO e GRPO: Ottimizzazione delle Policy nel Reinforcement Learning

Mattia Fornari - Mat. 731764
Gabriele Festa - Mat. 734870

Corso di Intelligenza Artificiale – A.A. 2024/2025

Reinforcement Learning

- Apprendimento tramite interazione con l'ambiente
- **Obiettivo**: massimizzare il return cumulativo atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

- Trial-and-error exploration
- Bilanciamento exploration vs. exploitation

Vantaggio

Advantage Function A(s, a): indica quanto un'azione specifica è migliore rispetto al comportamento medio della policy nello stato corrente. È definita come:

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

Interpretazione:

A(s,a)>0 → l'azione è meglio della media → “la rinforziamo”
A(s,a)<0 → l'azione è peggio della media → “la penalizziamo”
A(s,a)=0 → l'azione è in linea con la policy corrente

Vantaggio: Riduce la varianza nei policy gradient rispetto all'uso diretto di Q(s,a).

(Vanilla) Policy Gradient (PG)

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \right]$$

$$\hat{A}_t = G_t - V(s_t)$$

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t \right]$$

Gt è il return osservato nella traiettoria, A^_t è la stima del vantaggio

Pro e contro di PG

Pro:

- Ottimizzazione diretta della policy e semplicità di implementazione (e.g., algoritmo REINFORCE)
- Gestione efficace di spazi d'azione continui o ad alta dimensionalità e anche discreti
- Unbiased: stima corretta del gradiente vero

Contro:

- Alta varianza (Gt varia molto tra le traiettorie) e instabilità degli aggiornamenti
- Inefficienza in termini di interazioni con l'ambiente **(sample inefficiency):** ogni aggiornamento invalida i dati raccolti, richiedendo nuove interazioni con l'ambiente ad ogni step di ottimizzazione.
- Rischio di convergenza a ottimi locali e sensibilità a iperparametri, training lento e rumoroso

Trust Region Policy Optimization (TRPO)

- Metodo trust region
- Si massimizza una funzione obiettivo “surrogata” con un vincolo sull’aggiornamento della policy:

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \quad \text{soggetto a} \quad \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]] \leq \delta$$

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right]$$

Pro e contro di TRPO

Pro:

- Aggiornamenti più stabili e “sicuri” rispetto ai metodi PG
- Garanzia teorica di miglioramento monotono
- Adatto a problemi complessi e ambienti con reward rari / sparsi

Contro:

- Elevata complessità computazionale (second order optimization) e scalabilità difficile
- Compatibilità limitata con modelli complessi (difficile da applicare o molto lento)
- Difficoltà di implementazione, tuning e debug
- Limiti nell’esplorazione / possibile conservatività e sub-ottimalità

Metodi Actor-Critic

Combinazione di metodi value-based e policy-based:

- Actor: apprende la policy e seleziona le azioni
- Critic: stima il valore degli stati V(s) (o Q(s,a)) come baseline per il calcolo del vantaggio

Pro: riduce la varianza del gradiente, più sample-efficient di policy gradient puro, training più stabile

Contro: critic introduce complessità implementativa e ~2x costo computazionale (due reti) e ~2x memoria richiesta.

Esempi: **PPO**, A2C, A3C, SAC, DDPG

Proximity Policy Optimization (PPO) Schulman et al., 2017

Proximal Policy Optimization: algoritmo actor-critic on-policy di Deep RL che migliora TRPO sostituendo l'hard KL constraint con un clipped surrogate objective, permettendo first-order optimization (e.g., SGD/Adam).

Sia $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ il rapporto delle probabilità, con $r(\theta_{\text{old}}) = 1$. TRPO massimizza la funzione obiettivo:

$$L^{\text{CPI}}(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right]$$

Senza vincoli, massimizzare l'obiettivo sopra porta ad aggiornamenti illimitati della policy, causando un degrado delle prestazioni.

Proximity Policy Optimization (PPO)

Proximal Policy Optimization

- PPO-Clip
- PPO-Adaptive KL Penalty Coefficient

Entrambe mirano a limitare il cambiamento della policy.

PPO-Clip

Obiettivo: Limitare gli aggiornamenti che allontanano il rapporto rt(θ) da 1.

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

dove ε è un iperparametro per limitare il clipping (**clip range**), min{ · } = lower bound pessimistico (previene aggiornamenti eccessivi della policy).

Effetto: Previene e ignora aggiornamenti della policy troppo grandi, garantendo training stabile e assicurando che la policy non possa cambiare troppo drasticamente in un singolo aggiornamento.

PPO-Clip

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

- **Il clipping rimuove l'incentivo a spingere il rapporto r fuori dall'intervallo, imponendo cambiamenti graduali e migliorando la stabilità degli aggiornamenti.**
- Si nota che a seconda del segno del vantaggio A_t, si può stabilire un limite superiore o inferiore per l'aumento o decremento della probabilità di scelta di una certa azione.
- Aggiornamenti graduali e stabili, training robusto.

PPO-Clip

- Se $A_t > 0 \Rightarrow$ l'azione è meglio di quanto ci aspettavamo (più ricompensa del baseline) $\Rightarrow$ aumentiamo la probabilità, ma limitiamo a $r_t \leq 1+\varepsilon$ (max +20% se $\varepsilon=0.2$)
- Se $A_t < 0 \Rightarrow$ l'azione è peggiore della previsione $\Rightarrow$ diminuiamo la probabilità, ma limitiamo a $r_t \geq 1-\varepsilon$ (max -20% se $\varepsilon=0.2$)
- Se $A_t = 0 \Rightarrow$ l'azione ha reso esattamente come previsto $\Rightarrow$ nessun segnale di aggiornamento (termine nullo)

Nota: Upper bound $(1+\varepsilon)$: limita di quanto può essere aumentata la probabilità; Lower bound $(1-\varepsilon)$: limita di quanto può essere diminuire la probabilità

PPO-Adaptive KL Penalty Coefficient

Variante: si usa una penalità basata sulla divergenza di **K**ullback-**L**eibler invece del meccanismo di clipping.

$$L^{KLPEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta \, \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right]$$

Dove:

$$\mathbb{D}_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

- πθ è la policy corrente e πθ_old è la policy precedente;
- At è il vantaggio stimato;
- β è il coefficiente di penalità adattativo e KL è la divergenza di Kullback-Leibler.

PPO: implementazione pratica

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right]$$

dove $L_t^{VF} = (V_\theta(s_t) - V_t^{\text{target}})^2$ è l'errore quadratico della value function e S rappresenta il bonus di entropia, con c_1, c_2 coefficienti di pesatura.

Stima del vantaggio: in pratica si usano segmenti di esperienza di lunghezza fissa T campionati dalla policy corrente. At è stimato mediante **GAE** che non guarda oltre T e permette di bilanciare bias-varianza nella stima del vantaggio, riducendo la varianza delle stime del gradiente e rendendo l'apprendimento più stabile e affidabile.

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Algoritmo PPO

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Da PPO a GRPO per training LLM

Durante l'addestramento degli LLM per compiti di **ragionamento matematico complesso,** la fase di Reinforcement Learning successiva al fine-tuning supervisionato (SFT) si è rivelata cruciale per spingere ulteriormente le prestazioni.

Limitazioni di PPO per LLM training:

- **Onere computazionale e di memoria:** richiede una value function (critic) spesso di dimensioni comparabili alla policy, con elevato consumo di memoria GPU.
- **Complessità nell'addestramento:** stimare un valore accurato per ogni token è difficile quando il reward è sparso o assegnato solo all'output finale, rendendo il training del critic instabile e inefficiente.

Una possibile soluzione: GRPO

- **Innovazione DeepSeekMath**: introdotto come variante efficiente di PPO per superare i vincoli di memoria e calcolo, abilitando prestazioni SOTA anche su modelli open-source.

DeepSeekMath-RL 7B - Data, Pipeline & Performance

1. **DeepSeekMath Corpus & Pre-training**

   Corpus di 120B token matematici multilingue (Eng/Chi) estratti da Common Crawl mediante l'utilizzo di una pipeline iterativa con un classificatore fastText (addestrato inizialmente su OpenWebMath) per identificare ed estrarre pagine web matematiche, arricchita da annotazione umana. Viene dimostrato che il web pubblico contiene dati migliori di arXiv per il reasoning; supera corpus dedicati come MathPile.

2. **Pipeline di Addestramento e Strategia**

   Il modello base, DeepSeekMath-Base 7B, è inizializzato a partire da DeepSeek-Coder-Base-v1.5 7B. L'addestramento su codice (Code Training) migliora le capacità di ragionamento matematico e generale, offrendo un punto di partenza migliore rispetto ai LLM generici. Fase di **SFT:** Il modello base viene perfezionato con 776K esempi di istruzioni matematiche che includono soluzioni Chain-of-Thought (CoT), Program-of-Thought (PoT) e ragionamento integrato con strumenti.

DeepSeekMath-RL 7B - Data, Pipeline & Performance

Infine; SFT → Reinforcement Learning con **GRPO** → DeepSeekMath-RL 7B.

3. Performance

Le prestazioni di DeepSeekMath-Base 7B superano tutte quelle dei modelli open-source equivalenti e batte Minerva 540B (un modello 77 volte più grande) su benchmark matematici competitivi. Le prestazioni di DeepSeekMath-Instruct 7B superano tutti i modelli open-source da 7B a 70B e si avvicina alle prestazioni di GPT-4 e Gemini-Ultra sul benchmark MATH. Quando è permesso l'uso di strumenti (e.g., Python), il modello raggiunge un'accuratezza vicina al 60% su MATH, superando gli altri modelli open-source.

DeepSeekMath: GRPO

Funzione obiettivo:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[q \sim P(Q),\ \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t},\right.\right.$$
$$\left.\left.\text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})},\ 1-\epsilon,\ 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta\,\mathbb{D}_{KL}\left[\pi_\theta\|\pi_{ref}\right]\right\}$$

DeepSeekMath: Calcolo vantaggio in GRPO

Per ogni query q, viene campionato un gruppo di output {o1…oG} dalla policy π_old.La baseline non è stimata da un critic.

Outcome supervision

Il reward è un singolo scalare r_i per l'intera risposta o_i (es. 1 se corretto, 0 se errato).

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$$

Il vantaggio è identico per tutti i token della risposta.

Limite: non distingue quali passaggi del ragionamento sono corretti e quali no.

Process supervision

Il reward viene assegnato alla fine di ogni step di ragionamento, un vettore di reward per ogni step j della risposta i.

$$\tilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(R)}{\text{std}(R)}$$

$$\hat{A}_{i,t} = \sum_{index(j) \geq t} \tilde{r}_i^{index(j)}$$

Vantaggio: segnale di training più granulare

GRPO

GRPO rispetto a PPO permette di ricavare questi vantaggi:

• **Efficienza delle Risorse**: Rimuovendo il modello critico, si riduce drasticamente il consumo di memoria e il carico computazionale.

• **Allineamento con i Modelli di Reward (RM)**: PPO forza il RM a dare un valore/voto assoluto alla risposta. GRPO confronta le risposte all'interno del gruppo (calcolando la media), lavorando esattamente nel modo in cui il RM è stato addestrato (per confronti), rendendo il segnale più affidabile.

• **Regolarizzazione KL nella Loss**: A differenza del PPO che aggiunge una penalità KL nel reward (sporcando il segnale), il GRPO incorpora la divergenza KL direttamente nella funzione di perdita, utilizzando uno stimatore non distorto per garantire stabilità matematica.

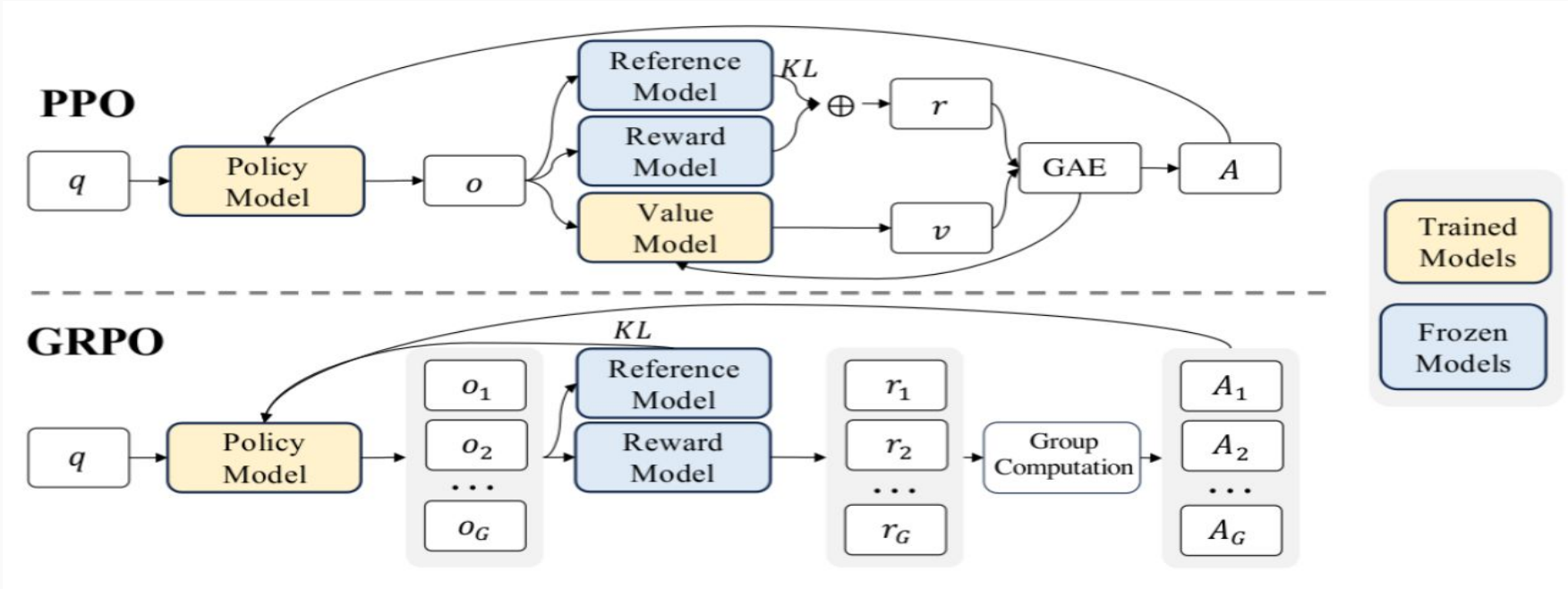
Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, ..., I **do**
    reference model $\pi_{ref} \leftarrow \pi_\theta$
    for step = 1, ..., M **do**
        Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
        Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
        Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
        Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
        Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
        for GRPO iteration = 1, ..., μ **do**
            Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
    Update r_φ through continuous training using a replay mechanism.

Output π_θ

PPO e GRPO per LLM training



RLHF e Scalabilità negli LLM: PPO vs GRPO

PPO

- Actor-Critic
- Caricamento simultaneo di quattro componenti: Actor, Reference, Reward Model e Critic
- Inefficienza di memoria VRAM

GRPO

- Elimina il Critic (ottimo quando la memoria è limitata)
- Scalabilità VRAM
- Molto efficiente con reward binari (per task matematici, codice)

PPO: Ottimizza l'efficienza dei dati (meno campioni) a costo della memoria.
GRPO: Ottimizza l'efficienza della memoria a costo dell'inferenza (richiede G generazioni per prompt).

GRPO permette di addestrare efficientemente modelli LLM molto più grandi mediante reinforcement learning, anche per task di ragionamento.

Altro su DeepSeekMath

- **Outcome Supervision RL con GRPO:** il vantaggio viene stimato in base alla deviazione standard e alla media dei rewards.
- **Process Supervision RL con GRPO**: stima del vantaggio in base alla somma dei rewards normalizzati.
- RL Iterativo con GRPO: generazione di nuovi dataset di addestramento per il modello di ricompensa basandosi sui risultati di campionamento della policy.
- Paradigma unificato: paradigma unificato per analizzare diversi metodi di addestramento come SFT, RFT, DPO, PPO e GRPO.

Altro su DeepSeekMath

- **Pre-training: L'addestramento su codice (DeepSeekCoder-Base-v1.5 7B) migliora il ragionamento matematico, mentre l'uso di paper ArXiv risulta inefficace.**
- **GRPO riduce le risorse necessarie (senza critic model) e supera le performance di modelli open-source molto più grandi (fino a 70B).**
- Robustezza vs Capacità: Il RL migliora la metrica Maj@K (consistenza/robustezza delle risposte corrette) ma non il Pass@K (capacità fondamentale), suggerendo che il RL "stabilizza" la conoscenza esistente piuttosto che insegnarne di nuova.
- Limiti: Il modello eccelle nel calcolo ma resta inferiore ai modelli proprietari (come GPT-4) in geometria e capacità few-shot.

Conclusioni

- **Caso d'uso tipico di PPO:** efficiente per RL generale (giochi, robotica) e RLHF classico (OpenAI, modelli di linguaggio);
- Complessità implementativa PPO: Medio-alta (richiede advantage estimation, due network: actor e critic, quattro nel caso di RLHF: actor, critic, reward, reference);
- Consumo elevato di VRAM;
- PPO utilizza un critic che stima un singolo valore atteso V(s).

Conclusioni

- **Caso d'uso tipico di GRPO**: Allineamento LLM con reward model/preferenze, meno complesso di PPO;
- Variante di PPO per migliorare la scalabilità e performance dell'RLHF;
- Non è necessario avere un modello critic caricato in memoria (e.g., in VRAM), riducendo significativamente le risorse di addestramento rispetto a PPO.
- Costo di sampling elevato: per ogni prompt GRPO campiona molte risposte candidate (maggior costo di inferenza GPU/latency).
- Dipendenza dal reward model: la qualità del segnale relativo dipende dalla capacità del reward model di ordinare correttamente le risposte.

Riferimenti

- Schulman et al., Proximal Policy Optimization Algorithms, arXiv:1707.06347.
- Shao et al., DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, arXiv:2402.03300.
- https://cameronrwolfe.substack.com/p/grpo